

Paxini Ring/Pinky 촉각 + LeRobot Pi0.5 한국어 가이드

1. 목표

이 문서는 다음 목표를 위해 작성했습니다.

1. Paxini GEN3 tactile sensor를 LeRobot SO101 follower에 붙입니다.
2. ring/pinky tactile 값을 LeRobot dataset feature로 저장합니다.
3. 저장된 dataset을 Pi0.5 policy가 읽을 수 있게 만듭니다.
4. 실제 센서가 없어도 PXSR CSV로 record/train 파이프라인을 검증합니다.
5. 공식 Hugging Face LeRobot과 현재 repo의 차이를 한눈에 확인합니다.

현재 추천 tactile 표현은 (2, 9, 9) 입니다.

```
channel 0: ring proximal 77 taxels -> 9x9, four corners padded with 0
channel 1: pinky proximal 77 taxels -> 9x9, four corners padded with 0
final shape: (2, 9, 9)
```

이렇게 하면 Pi0.5 tactile CNN의 첫 layer가 `Conv2d(in_channels=2, kernel_size=3)` 가 되어 ring/pinky를 처음부터 같이 볼 수 있습니다.

2. 전체 구조

```
Paxini sensor or PXSR CSV
  |
  v
PaxiniReader
  |
  v
154 taxel vector
  |
  +--> heatmap      -> observation.tactile.primary: (64, 64)
  |
  +--> stacked_9x9  -> observation.tactile.primary: (2, 9, 9)
                        recommended

S0101 follower state -----+
Camera image -----+--> LeRobotDataset
Tactile observation -----+
```

```

Leader action -----+
Task text -----+

LeRobotDataset
|
v
Pi0.5 policy
|
+--> image encoder
+--> state/action processor
+--> tactile CNN token encoder
+--> action expert

```

3. 공식 LeRobot 대비 현재 repo 변경점

공식 기준은 `origin/main` 입니다.

영역	공식 LeRobot	현재 repo 변경
Robot type	기본 SO100/SO101 follower만 있음	<code>paxini_so101_follower</code> 추가
Tactile feature type	기본 feature에 tactile 없음	<code>FeatureType.TACTILE</code> , <code>OBS_TACTILE</code> 경로 추가
Robot observation	joint/camera 중심	<code>observation.tactile.primary</code> 추가
Pi0/Pi0.5 policy	image/state/action 중심	tactile prefix token encoder 추가
Tactile encoder	없음	CNN/attention CNN encoder 추가
Dataset feature 변환	image/state/action 중심	tactile feature를 float32 tensor로 저장
Processor	tactile batch/validation 없음	tactile batch dimension/validation processor 추가
Sensor reader	없음	serial, mock, PXSR CSV reader 추가
CSV smoke test	없음	<code>make_pi05_smoke_dataset.py</code> 추가

4. 변경 파일 요약

Paxini 전용 패키지

파일	역할
<code>src/lerobot_robot_paxini/__init__.py</code>	Paxini robot/config export
<code>config_paxini_so101.py</code>	tactile port, taxel 수, representation 설정
<code>paxini_reader.py</code>	serial/mock/CSV tactile reader
<code>paxini_so101.py</code>	SO101 follower에 tactile observation 추가
<code>tactile_render.py</code>	154 taxel -> heatmap 또는 2x9x9 변환
<code>types.py</code>	<code>PaxiniSample</code> dataclass
<code>make_pi05_smoke_dataset.py</code>	센서 없이 LeRobot dataset 생성
<code>README.md</code>	짧은 사용 가이드
<code>PAXINI_PI05_GUIDE.ko.md</code>	이 문서

LeRobot core 변경

파일	변경 내용
<code>src/lerobot/robots/__init__.py</code>	<code>lerobot_robot_paxini</code> 자동 import로 robot type 등록
<code>src/lerobot/utils/constants.py</code>	tactile observation prefix 추가
<code>src/lerobot/utils/feature_utils.py</code>	tactile feature를 dataset/policy feature로 변환
<code>src/lerobot/configs/types.py</code>	tactile feature type 추가
<code>src/lerobot/configs/policies.py</code>	tactile feature 접근용 helper 추가
<code>src/lerobot/processor/batch_processor.py</code>	tactile tensor/array에 batch dimension 추가
<code>src/lerobot/processor/tactile_processor.py</code>	tactile shape validation/temporal filter
<code>src/lerobot/processor/__init__.py</code>	tactile processor export
<code>src/lerobot/policies/tactile/encoder.py</code>	tactile CNN token encoder 추가
<code>src/lerobot/policies/pi0/configuration_pi0.py</code>	Pi0 tactile config 추가
<code>src/lerobot/policies/pi0/modeling_pi0.py</code>	Pi0 tactile token 입력 추가

파일	변경 내용
src/lerobot/policies/pi05/configuration_pi05.py	Pi0.5 tactile config 추가
src/lerobot/policies/pi05/modeling_pi05.py	Pi0.5 tactile token 입력 추가

5. 센서 데이터 해석

현재 확인된 연결은 ring proximal과 pinky proximal입니다.

```
Ring proximal : 77 taxels
Pinky proximal : 77 taxels
Total          : 154 taxels
```

PXSR CSV 컬럼 예시는 다음과 같습니다.

```
3-0-NxN-Z[0] ... 3-0-NxN-Z[76]
4-0-NxN-Z[0] ... 4-0-NxN-Z[76]
```

이 문서에서는 3-0 을 ring, 4-0 을 pinky로 취급합니다.

6. 왜 (2, 9, 9) 가 좋은가

처음 구현은 154개 값을 (64, 64) heatmap으로 흘뿌리는 방식이었습니다. 이 방식도 동작하지만 작은 tactile array를 큰 이미지처럼 보간하기 때문에 실제 센서 배열 구조가 흐려질 수 있습니다.

반면 (2, 9, 9) 는 다음 장점이 있습니다.

- ring과 pinky가 서로 다른 channel로 유지됩니다.
- 77개 taxel의 원래 격자성이 보존됩니다.
- 네 모서리만 0으로 padding하므로 불필요한 보간이 없습니다.
- CNN kernel 3x3 가 인접 taxel 관계를 직접 볼 수 있습니다.
- Pi0.5 앞단에서 tactile token으로 넣기 쉽습니다.

현재 구현은 depthwise convolution이 아닙니다. 첫 convolution이 `Conv2d(2, 32, kernel_size=3, padding=1)` 처럼 동작해서 두 손가락 channel을 처음부터 섞어 볼 수 있습니다.

7. 환경 설정

repo 루트에서 실행합니다.

```
cd /mnt/c/Users/dlehg/conference/vtla
conda activate lerobot
export PYTHONPATH=/mnt/c/Users/dlehg/conference/vtla/lerobot/src:$PYTHONPATH
```

포트 확인:

```
ls -l /dev/serial/by-id/
python -m serial.tools.list_ports -v
```

Windows PXSR 앱이 COM 포트를 잡고 있으면 WSL에서 동시에 읽을 수 없습니다.

8. 센서 없이 CSV로 dataset 만들기

PXSR CSV가 있으면 실제 tactile sensor 없이도 LeRobot dataset을 만들 수 있습니다.

```
conda run -n lerobot python -m lerobot_robot_paxini.make_pi05_smoke_dataset \
  --repo-id local/paxini_pi05_smoke_2x9x9 \
  --root /tmp/lerobot_paxini_smoke_2x9x9 \
  --csv /mnt/c/Users/dlehg/AppData/Roaming/pxsr-gen3/DataLogging/2026-05-16-182429.csv \
  --frames 32 \
  --fps 10 \
  --tactile-representation stacked_9x9
```

정상 생성 결과:

```
Created dataset repo_id=local/paxini_pi05_smoke_2x9x9
Dataset root=/tmp/lerobot_paxini_smoke_2x9x9
Frames=32, tactile_shape=(2, 9, 9)
```

shape 확인:

```
conda run -n lerobot python -c "\
from lerobot.datasets import LeRobotDataset; \
ds=LeRobotDataset('local/paxini_pi05_smoke_2x9x9', root='/tmp/lerobot_paxini_smoke_2x9x9'); \
item=ds[0]; \
print(len(ds)); \
print(item['observation.state'].shape); \
print(item['observation.tactile.primary'].shape); \
```

```
print(item['observation.images.front'].shape); \
print(item['action'].shape)"
```

기대 출력:

```
32
torch.Size([6])
torch.Size([2, 9, 9])
torch.Size([3, 224, 224])
torch.Size([6])
```

9. 실제 센서로 teleoperate

```
lerobot-teleoperate \
  --robot.type=paxini_so101_follower \
  --robot.port=/dev/ttyACM_FOLLOWER \
  --robot.id=so101_paxini \
  --robot.tactile_port=/dev/ttyACM_SENSOR \
  --robot.tactile_num_taxels=154 \
  --robot.tactile_representation=stacked_9x9 \
  --teleop.type=so101_leader \
  --teleop.port=/dev/ttyACM_LEADER \
  --teleop.id=so101_leader \
  --fps=30 \
  --display_data=true
```

센서 없이 robot code path만 확인하려면:

```
lerobot-teleoperate \
  --robot.type=paxini_so101_follower \
  --robot.port=/dev/ttyACM_FOLLOWER \
  --robot.id=so101_paxini \
  --robot.tactile_mock=true \
  --robot.tactile_num_taxels=154 \
  --robot.tactile_representation=stacked_9x9 \
  --teleop.type=so101_leader \
  --teleop.port=/dev/ttyACM_LEADER \
  --teleop.id=so101_leader \
  --fps=30
```

CSV를 센서 대신 replay하려면:

```
lerobot-teleoperate \
  --robot.type=paxini_so101_follower \
  --robot.port=/dev/ttyACM_FOLLOWER \
  --robot.id=so101_paxini \
  --robot.tactile_csv_path=/mnt/c/Users/dlehg/AppData/Roaming/pxsr-gen3/DataLogging/2026-05-16
```

```
--robot.tactile_num_taxels=154 \
--robot.tactile_representation=stacked_9x9 \
--teleop.type=so101_leader \
--teleop.port=/dev/ttyACM_LEADER \
--teleop.id=so101_leader \
--fps=30
```

10. record

record의 프레임 흐름은 다음입니다.

```
robot.get_observation()
teleop.get_action()
robot.send_action()
dataset.add_frame()
```

실제 기록 명령:

```
lerobot-record \
--robot.type=paxini_so101_follower \
--robot.port=/dev/ttyACM_FOLLOWER \
--robot.id=so101_paxini \
--robot.tactile_port=/dev/ttyACM_SENSOR \
--robot.tactile_num_taxels=154 \
--robot.tactile_representation=stacked_9x9 \
--robot.cameras='{front: {type: opencv, index_or_path: 0, width: 640, height: 480, fps: 30}}' \
--teleop.type=so101_leader \
--teleop.port=/dev/ttyACM_LEADER \
--teleop.id=so101_leader \
--dataset.repo_id=dlehg/so101_paxini_ring_pinky \
--dataset.single_task="pick up the object using ring and pinky tactile feedback" \
--dataset.fps=30 \
--dataset.num_episodes=5 \
--dataset.episode_time_s=30 \
--dataset.reset_time_s=10 \
--dataset.push_to_hub=false
```

처음에는 `num_episodes=1` 로 시작하는 것을 권장합니다.

11. replay

replay는 학습이 아니라 저장된 action을 다시 보내는 검증입니다.

```
lerobot-replay \
  --robot.type=paxini_so101_follower \
  --robot.port=/dev/ttyACM_FOLLOWER \
  --robot.id=so101_paxini \
  --robot.tactile_port=/dev/ttyACM_SENSOR \
  --robot.tactile_num_taxels=154 \
  --robot.tactile_representation=stacked_9x9 \
  --dataset.repo_id=dlehg/so101_paxini_ring_pinky_YYYYMMDD_HHMMSS \
  --dataset.episode=0
```

12. Pi0.5 train

LeRobot에서 Pi0.5 policy type은 **pi05** 입니다.

작은 smoke train:

```
lerobot-train \
  --dataset.repo_id=local/paxini_pi05_smoke_2x9x9 \
  --dataset.root=/tmp/lerobot_paxini_smoke_2x9x9 \
  --policy.type=pi05 \
  --policy.paligemma_variant=gemma_300m \
  --policy.action_expert_variant=gemma_300m \
  --policy.dtype=bfloat16 \
  --policy.device=cuda \
  --policy.tactile_encoder_type=cnn \
  --policy.train_expert_only=true \
  --policy.push_to_hub=false \
  --batch_size=1 \
  --steps=1 \
  --num_workers=0 \
  --eval_freq=0 \
  --save_freq=1 \
  --wandb.enable=false
```

실제 fine-tune:

```
lerobot-train \
  --dataset.repo_id=dlehg/so101_paxini_ring_pinky_YYYYMMDD_HHMMSS \
  --policy.path=lerobot/pi05_base \
  --policy.dtype=bfloat16 \
  --policy.device=cuda \
  --policy.tactile_encoder_type=cnn \
  --policy.train_expert_only=true \
  --policy.push_to_hub=false \
  --batch_size=1 \
  --steps=3000 \
  --save_freq=1000 \
  --eval_freq=0 \
  --wandb.enable=false
```

13. rollout

학습된 policy로 실제 로봇을 움직일 때는 `lerobot-rollout` 을 씁니다.

```
lerobot-rollout \
  --strategy.type=base \
  --policy.path=outputs/train/YYYY-MM-DD/HH-MM-SS_pi05/checkpoints/last/pretrained_model \
  --inference.type=rtc \
  --inference.rtc.execution_horizon=10 \
  --robot.type=paxini_so101_follower \
  --robot.port=/dev/ttyACM_FOLLOWER \
  --robot.id=so101_paxini \
  --robot.tactile_port=/dev/ttyACM_SENSOR \
  --robot.tactile_num_taxels=154 \
  --robot.tactile_representation=stacked_9x9 \
  --task="pick up the object using ring and pinky tactile feedback" \
  --duration=60
```

14. 검증 체크리스트

Dataset 생성 후

```
observation.state:          torch.Size([6])
observation.tactile.primary torch.Size([2, 9, 9])
observation.images.front:   torch.Size([3, 224, 224])
action:                    torch.Size([6])
```

Tactile CNN 확인

```
from lerobot.policies.tactile import TactileTokenEncoder
import torch

enc = TactileTokenEncoder("cnn", input_shape=(2, 9, 9), feature_dim=32)
y = enc(torch.randn(4, 2, 9, 9))
print(y.shape)
print(enc.backbone.conv1.in_channels)
```

기대 출력:

```
torch.Size([4, 1, 32])
2
```

15. 자주 확인할 것

- `tactile_num_taxels` 는 ring+pinky proximal 기준 154 입니다.
- 158 로 들어가면 total force byte까지 taxel로 잘못 센 가능성이 큽니다.
- `stacked_9x9` 는 154개가 정확히 들어올 때 가장 깔끔합니다.
- 실제 sensor와 PXSR Windows 앱은 같은 COM port를 동시에 잡을 수 없습니다.
- 처음에는 CSV smoke dataset으로 train 경로를 먼저 확인한 뒤 실제 record로 넘어갑니다.

16. 추천 순서

1. PXSR CSV로 `stacked_9x9` smoke dataset 생성
2. dataset shape 확인
3. tactile CNN forward 확인
4. Pi0.5 smoke train 1 step
5. 실제 센서 연결
6. `lerobot-teleoperate` 로 로봇과 tactile 연결 확인
7. `lerobot-record` 로 1 episode 기록
8. `lerobot-replay` 로 action 검증
9. 충분한 episode 기록
10. Pi0.5 fine-tune
11. `lerobot-rollout` 으로 실제 실행